COURSE LAB GUIDE · 32 LABS · 7 PARTS

Disaster Recovery Lab Guide

One lab per video lesson. Build a small environment, analyse it like a business, break it on purpose, and prove you can bring it back inside the time and data-loss window you set.

RHYTHM

Learn, recover, test — every part ends with something proven, not just planned.

EVERY LAB

Objective, prerequisites, full walkthrough, checks, reflection and the pitfall to avoid.

LEVEL

Beginner. You need to open a terminal and create a VM; every command is given in full.

How to use this guide

Do each lab after its video. Labs build on each other: the threat catalog from Lab 2 feeds the risk register in Lab 7, the business impact analysis in Lab 6 sets the objectives you measure against in Lab 24, and the immutable backup from Lab 15 is what saves you in Lab 23. Keep every artefact.

Parts I and II are mostly analysis and writing; they use worksheets. Parts III to VII are mostly hands-on in the lab environment you build in Lab 3. Where a lab asks for a partner, a colleague or classmate makes it far more realistic — but each lab can be done alone.

A lab is finished when every item under *Check your work* is true and you have written the result in your lab log — not when the commands have run.

The lab environment

Three Ubuntu 24.04 virtual machines on any hypervisor that supports snapshots. The protected application is Gitea (a self-hosted Git service) backed by PostgreSQL, with a heartbeat that writes one row per second so you can measure data loss to the second.

HOST	ROLE	RUNS
`prod-a`	Primary site	Gitea, PostgreSQL 16, heartbeat writer (Docker Compose). Second disk for local backups.
`dr-b`	Recovery target	Docker only. Everything on it arrives through a recovery procedure, never by hand.
`vault`	Off-site backup store	MinIO object storage with an object-locked bucket for immutable copies.
`workstation`	You	SSH, psql, restic, the MinIO client (mc), and your lab log.
`cloud`	Lab 22 only	Any provider with object storage and VMs. Set a budget alert before you start.

Conventions

— **Snapshot, break, revert.** Snapshot every VM before a destructive step. Revert afterwards unless the next lab says to keep the state.

— **Keep a lab log.** Dated entries with commands, timestamps and outcomes. A recovery you cannot repeat is not a recovery — and later labs reuse what you record.

— **Where commands run.** On prod-a as user `lab` in `~/fieldapp`, unless the step names another host. Hostnames resolve through `/etc/hosts`. Values in <angle brackets> are yours to fill in.

— **Lab-only secrets.** Passwords printed here exist only so the commands work. Never reuse them anywhere real.

— **Shorthand.** `q "SQL"` runs SQL against the production database; you define it in Lab 3.

Contents

PART I

Foundations

01 Inventory what you would actually lose
02 Build a threat catalog
03 Build your disaster recovery lab
04 Set RTO and RPO for every system
05 Write a recovery approach and its dependencies

PART II

Analysis, Strategy and Planning

06 Conduct a business impact analysis
07 Produce a prioritised risk register
08 Draft a tiered DR strategy
09 Write an executable recovery procedure
10 Design a recovery site approach
11 Build a cost justification

PART III

Backup

12 Implement full-plus-incremental backups
13 Architect for 3-2-1 with an immutable copy
14 Prove an application-consistent backup restores
15 Configure an immutable backup and attack it
16 Run a full restore test against the clock

PART IV

Replication and Availability

17 Set up replication and measure the data-loss window
18 Show what high availability cannot save

PART V

Recovery Mechanics

19 Perform a point-in-time database recovery
20 Rebuild a host and restore its workload
21 Recover a multi-tier application end to end
22 Recover a workload into the cloud
23 Recover cleanly from ransomware

PART VI

Testing and Sustaining

24 Run a full failover test
25 Fix the gaps and retest
26 Change the environment, then the plan
27 Run a people-first tabletop exercise
28 Extend the plan into continuity

PART VII

Execution and Mastery

29 Execute a full recovery from a declared disaster
30 Diagnose staged recovery failures
31 Audit your program against the failure catalog
32 Assess your skills and plan the next 90 days

PART I

Foundations

Vocabulary, threats, the lab itself, the two numbers that govern everything, and the method behind every recovery.

Lab 01	Inventory what you would actually lose
Lab 02	Build a threat catalog
Lab 03	Build your disaster recovery lab
Lab 04	Set RTO and RPO for every system
Lab 05	Write a recovery approach and its dependencies

Inventory what you would actually lose

OBJECTIVE

Find out, for one system you rely on, the distance between having copies and being able to recover.

YOU NEED

One real system you depend on — at work or at home. No lab environment yet.

YOU WILL PRODUCE

A loss inventory with a recovery path, a copy age and an honest time estimate for every item.

PROCEDURE

01 Pick your most important system: the one whose loss would hurt most this week. Write one sentence on what it does for you or the business.

02 Imagine it disappeared right now — hardware, account and local copies gone. List everything that goes with it: data, configuration, credentials, integrations, history.

03 For each item, record where any copy lives and when it was last updated. Write “none” where there is none.

04 Write how you would get each item back without the original system. Mark every step that relies on memory or on one person.

05 Estimate how long the whole recovery would take and how much recent work would be lost. These are your first, informal RTO and RPO.

06 Circle every item where a copy exists but you have never restored from it. That gap is what this course closes.

WORKSHEET

ITEM LOST	WHERE A COPY LIVES	COPY AGE	HOW I WOULD RECOVER IT	EST. TIME

CHECK YOUR WORK

- [ ] Every item has a recovery path or an explicit “none”.
- [ ] You separated “a copy exists” from “I have restored it”.
- [ ] You have one overall time-to-recover and one data-loss estimate.

WATCH FOR

Listing only data. Licences, MFA devices, DNS records and the password to the backup account are what stall real recoveries.

REFLECT

Which item on your list would take longest to recover, and why?

What surprised you about the difference between having a copy and being able to restore it?

Build a threat catalog

OBJECTIVE

Ground your preparation in how systems really fail, rated by likelihood and impact rather than by fear.

YOU NEED

The system from Lab 1 (or, later, your lab environment).

YOU WILL PRODUCE

A threat catalog of at least twelve threats across four categories, each rated and mapped to a recovery capability.

PROCEDURE

01 Draw the system’s components on one page: hosts, storage, network, accounts, providers, people.

02 Brainstorm threats in four categories — natural, technical, human, malicious. Aim for at least three per category.

03 Add the boring ones explicitly: disk failure, full disk, expired certificate, accidental delete, bad configuration push, corrupted file that nobody notices for a week.

04 Rate likelihood 1–5 over one year and impact 1–5 to the business. Multiply for a score.

05 For the top three, write one cascade: what does the first failure trigger next?

06 Rewrite your top threats as capabilities you need — “restore any file from yesterday”, “run from another site” — rather than as scenarios.

WORKSHEET

THREAT	CATEGORY	LIKELIHOOD 1–5	IMPACT 1–5	SCORE	CAPABILITY NEEDED

CHECK YOUR WORK

- [ ] All four categories appear.
- [ ] At least two human-error threats are listed.
- [ ] Every top-five threat maps to a capability.

WATCH FOR

Rating ransomware and floods 5/5 and disk failure 1/5. Mundane failures happen far more often; rate on evidence, not drama.

REFLECT

Which threat did you rate highest, and what evidence supports that rating?

Which single capability would cover the most threats on your list?

Build your disaster recovery lab

OBJECTIVE

Stand up an application worth protecting, with data that changes every second, and separate places to back it up to and recover it into.

YOU NEED

A hypervisor with snapshots (Proxmox, VirtualBox, VMware or Hyper-V), about 12 GB free RAM, Ubuntu 24.04 ISO.

YOU WILL PRODUCE

A running Gitea stack on prod-a with a heartbeat, an empty dr-b, MinIO on vault, baseline snapshots and the first lab-log entries.

PROCEDURE

01 Create three VMs — prod-a, dr-b, vault — each 2 vCPU, 4 GB RAM, 40 GB disk. Give prod-a a second 40 GB disk for local backups. Assign static IPs and add all three to /etc/hosts on every machine, including your workstation.

02 Install the tooling on all three VMs.

```
sudo apt update && sudo apt install -y docker.io docker-compose-v2 \
  postgresql-client restic
sudo usermod -aG docker $USER    # log out and back in
```

03 On prod-a, create ~/fieldapp/compose.yaml and start the stack with “docker compose up -d”. The heartbeat service inserts a timestamped row every second.

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: gitea
      POSTGRES_PASSWORD: lab-only-pass
      POSTGRES_DB: gitea
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    restart: unless-stopped
  app:
    image: gitea/gitea:1.22
    environment:
      GITEA__database__DB_TYPE: postgres
      GITEA__database__HOST: db:5432
      GITEA__database__NAME: gitea
      GITEA__database__USER: gitea
      GITEA__database__PASSWD: lab-only-pass
    ports: ["3000:3000", "2222:22"]
    volumes: ["giteadata:/data"]
    depends_on: [db]
    restart: unless-stopped
  heartbeat:
    image: postgres:16
    environment: {PGPASSWORD: lab-only-pass}
    depends_on: [db]
    restart: unless-stopped
    command: >
      sh -c 'until psql -h db -U gitea -d gitea -qc "CREATE TABLE IF NOT EXISTS
      dr_heartbeat (id bigserial PRIMARY KEY, ts timestamptz DEFAULT now())";
      do sleep 2; done; while true; do psql -h db -U gitea -d gitea -qc
      "INSERT INTO dr_heartbeat DEFAULT VALUES"; sleep 1; done'
volumes:
  pgdata: {}
  giteadata: {}
```

04 Define the q shorthand used throughout this guide, then confirm the heartbeat is running.

```
echo "alias q='docker compose -f ~/fieldapp/compose.yaml exec -T db psql -U gitea -d
gitea -c'" >> ~/.bashrc
source ~/.bashrc
q "SELECT count(*), max(ts) FROM dr_heartbeat;"
```

05 Open http://prod-a:3000 and finish the installer. Create an admin user, two repositories, a few issues and an attachment. This is the data you will protect.

06 On vault, start MinIO.

```
docker run -d --name minio --restart unless-stopped \
  -p 9000:9000 -p 9001:9001 -v /srv/minio:/data \
  -e MINIO_ROOT_USER=labadmin -e MINIO_ROOT_PASSWORD=lab-only-secret \
  quay.io/minio/minio server /data --console-address ":9001"
```

07 Leave dr-b empty apart from Docker. It is your recovery target; nothing should ever be copied there by hand.

08 Shut down all three VMs and take a snapshot of each named “baseline”. Boot them again and check the heartbeat resumes on its own.

CHECK YOUR WORK

- [ ] Running the count query twice a few seconds apart shows the count rising.
- [ ] Reverting prod-a to baseline boots a working stack without manual steps.
- [ ] Your lab log records IPs, versions, and where every credential is kept.

WATCH FOR

Putting all three VMs on one physical host and one datastore without noting it. That is fine for a lab — but write it down as a shared dependency; Lab 10 asks for it.

REFLECT

Which parts of this lab environment share a dependency, and what would that mean in a real disaster?

If you had to rebuild this lab from your log alone, what would be missing?

Set RTO and RPO for every system

OBJECTIVE

Put the two governing numbers on each part of your lab, then face what you could actually achieve today.

YOU NEED

Your running lab from Lab 3. Treat Gitea as your team’s source-control service.

YOU WILL PRODUCE

An objectives table with target and estimated actual RTO and RPO, and the gap, for each system.

PROCEDURE

01 List each recoverable unit: the Gitea application, the PostgreSQL database, the repository files, the MinIO backup store, and your lab log.

02 Assign an RTO and an RPO to each, with one sentence of justification in business terms.

03 Estimate what you could achieve right now with what exists. You have no backups yet — “unrecoverable” is a valid and honest answer.

04 Compute the gap between objective and actual for each.

05 For one system, describe what it would take to halve its RTO, and separately to halve its RPO. Note that the two buy different things.

06 Keep this table. Labs 16, 17 and 24 replace your estimates with measurements.

WORKSHEET

SYSTEM	RTO TARGET	RPO TARGET	ACTUAL RTO (EST.)	ACTUAL RPO (EST.)	GAP

CHECK YOUR WORK

- [ ] Different systems have different objectives.
- [ ] Every objective has a reason tied to impact, not to the tooling you happen to have.
- [ ] Actuals are clearly marked as estimates.

> **WATCH FOR**
>
> Setting RPO from the backup schedule you already run. The objective comes first; the schedule is then made to meet it.

REFLECT

Which system has the largest gap between objective and actual?

Why would halving RTO and halving RPO require different investments?

Write a recovery approach and its dependencies

OBJECTIVE

Apply the core method — identify, define, design, prove — to a scenario where the things you normally rely on are gone too.

YOU NEED

The scenario below and your lab diagram.

YOU WILL PRODUCE

A one-page recovery approach and a list of every dependency the recovery itself needs.

SCENARIO

02:40. The hypervisor running prod-a has a storage controller failure; the VM's disk is unreadable. The admin who built the system is on a flight until 14:00. The lab log lives on a shared drive served from the same host.

PROCEDURE

01 Write one sentence defining what "recovered" means for the service — what a user must be able to do — not for the data.

02 Write the approach in four stages: what matters, how fast and how complete, the recovery path, and how you will prove it worked.

03 List every dependency the recovery requires: a target host, network, DNS, credentials, backup copies, installers, the written procedure, and people.

04 Mark each dependency that is also unavailable in this scenario. Every "yes" needs an alternative.

05 Rewrite the approach so someone who has never seen the system could follow it at 3 a.m., changing one thing at a time.

06 Find the step where you would be most tempted to improvise. Add an explicit stop-and-verify there.

WORKSHEET

DEPENDENCY	NEEDED FOR	DOWN IN THIS SCENARIO?	ALTERNATIVE

CHECK YOUR WORK

- [ ] The approach ends with a user-visible service check, not “restore completed”.
- [ ] Credentials and the procedure document are on the dependency list.
- [ ] No step assumes the absent admin.

WATCH FOR

Forgetting the recovery’s own needs. The backup system’s password often lives in a password manager hosted on the system that just failed.

REFLECT

Which dependency of the recovery itself had you not thought of before?

How would your approach change if you were the only responder?

PART II

Analysis, Strategy and Planning

Decide what deserves protection and how much, where recovery happens and how to pay for it — then write it so a stranger can execute it.

Lab 06	Conduct a business impact analysis
Lab 07	Produce a prioritised risk register
Lab 08	Draft a tiered DR strategy
Lab 09	Write an executable recovery procedure
Lab 10	Design a recovery site approach
Lab 11	Build a cost justification

Conduct a business impact analysis

OBJECTIVE

Derive recovery objectives from what an outage actually costs the business, rather than from technical convenience.

YOU NEED

Your objectives table from Lab 4.

YOU WILL PRODUCE

A BIA table with impact over time, derived RTO/RPO and a criticality tier for every system.

PROCEDURE

01 Define four to six business processes your lab supports — for example: developers push code, CI pulls code, issue tracking, release tagging, audit history.

02 Map each process to every lab system it depends on, including indirect ones such as DNS, identity and the backup store.

03 Estimate the impact of losing each process for one hour, one day and one week. Use plain words and rough money where you can.

04 Score each across the impact dimensions: financial, operational, reputational, legal and regulatory.

05 Set RTO at the point where impact becomes unacceptable, and RPO at how much lost work is tolerable. Revise your Lab 4 numbers and note what changed.

06 Sort systems into tiers: Tier 1 back within hours, Tier 2 within a day, Tier 3 eventually.

07 Look for a “low priority” system that a Tier 1 process quietly depends on. Promote it, or record why not.

WORKSHEET

PROCESS	DEPENDS ON	1 HOUR	1 DAY	1 WEEK	RTO	RPO	TIER

CHECK YOUR WORK

- [ ] Every RTO and RPO traces back to an impact row.
- [ ] At least one dependency changed a system’s tier.
- [ ] Changes from Lab 4 are recorded.

WATCH FOR

Accepting the most optimistic estimate. Ask “what happens on day three?” — impact usually compounds, and owners underrate it.

REFLECT

Which system changed tier once you mapped dependencies?

How did deriving objectives from impact change your Lab 4 numbers?

Produce a prioritised risk register

OBJECTIVE

Turn threats and assets into ranked risks, and hunt down every single point of failure.

YOU NEED

Your threat catalog (Lab 2) and BIA (Lab 6).

YOU WILL PRODUCE

A risk register with treatments, plus a list of single points of failure including at least one that is not technical.

PROCEDURE

01 Pair threats with assets to form risks, written as “threat → asset → consequence”: disk failure on prod-a destroys pgdata.

02 Rate likelihood from evidence — hardware age, change frequency, past incidents. Take impact from the asset’s BIA tier.

03 Sort by score. The top five get your attention first.

04 Hunt single points of failure: walk every component and ask “if only this disappears, what stops?” Include people, credentials and the single hypervisor.

05 Choose a treatment for each risk — accept, mitigate, transfer or avoid — and write the DR work each mitigation implies.

06 Remove false precision. If you cannot honestly tell a 3 from a 4, use high, medium and low.

WORKSHEET

ID	RISK (THREAT → ASSET)	L	I	SCORE	SPOF?	TREATMENT	DR ACTION

CHECK YOUR WORK

- [ ] At least three single points of failure, one of them non-technical.
- [ ] Every accepted risk has a named owner.
- [ ] Each top risk maps to a later lab where you address it.

WATCH FOR

A register built for the auditor: forty rows, a colourful heat map and no actions. Ten real risks with treatments are worth more.

REFLECT

Which single point of failure worries you most, and is it technical?

Which risk did you choose to accept, and who should sign off on that?

Draft a tiered DR strategy

OBJECTIVE

Match protection to value: one approach per tier, costed and agreed, before you touch any technology.

YOU NEED

BIA (Lab 6) and risk register (Lab 7).

YOU WILL PRODUCE

A one-page strategy assigning every system an approach, a recovery location and an estimated cost.

PROCEDURE

01 List your tiers and their objectives from the BIA.

02 For each tier, pick a point on the spectrum: backup to off-site, backup plus warm standby, replication to a ready site, or active redundancy.

03 Assign every lab system to its tier’s approach. Justify any exception in one line.

04 Estimate the monthly cost of each approach: storage, standby compute, licences and operator hours.

05 Decide build, buy or cloud for the recovery capability of each tier.

06 Ask: would the organisation fund and maintain this for three years? Cut what it would not.

07 Write the strategy as one page with a version and date: objectives, approach per tier, and what is deliberately left minimal.

WORKSHEET

SYSTEM	TIER	RTO / RPO	APPROACH	RECOVERY LOCATION	MONTHLY COST

CHECK YOUR WORK

- [ ] The Tier 1 approach can meet its RPO on paper — a nightly backup cannot meet a 15-minute RPO.
- [ ] Something is deliberately protected less.
- [ ] Someone who was not involved can read and follow it.

WATCH FOR

Gold-plating the trivial: replicating the wiki in real time while the database gets a nightly dump.

REFLECT

Where is your strategy deliberately cheap, and are you comfortable with that?

What would break first if the budget were cut by half?

Write an executable recovery procedure

OBJECTIVE

Write one procedure specific enough that a stressed, unfamiliar person could follow it without you.

YOU NEED

Your strategy (Lab 8). Ideally a second person to test it.

YOU WILL PRODUCE

A recovery procedure for the PostgreSQL database onto dr-b, tested cold by someone else.

PROCEDURE

01 Pick the database restored onto dr-b from the latest off-site backup. You will build this backup in Part III — write the procedure now as intended and correct it later.

02 Write the activation criteria: which conditions trigger this procedure and who may declare it.

03 List prerequisites: access, credentials and where they are kept, tools, and what must already be running.

04 Write numbered steps. Each has one action, the exact command, the expected result, and what to do if the result differs.

05 Add a roles box: primary responder, backup, and what happens if both are unreachable.

06 Add contacts, and say where this document lives when the wiki is down. Print a copy and store one off-site.

07 Hand it to someone else (or yourself after a week away) to follow cold. Every question they ask is a missing line.

WORKSHEET

#	ACTION / COMMAND	EXPECTED RESULT	IF NOT

CHECK YOUR WORK

- ☐ No step says “configure as appropriate”.
- ☐ Someone else completed it without asking you anything.
- ☐ A copy exists outside the lab.

WATCH FOR

Writing policy instead of procedure. “Restore the database per standards” is useless at 3 a.m.

REFLECT

What questions did your tester ask that you had not anticipated?

Which step would be hardest to follow while stressed and tired?

Design a recovery site approach

OBJECTIVE

Decide where recovery happens, what that choice does to your RTO, and what the recovery site secretly shares with production.

YOU NEED

Your strategy (Lab 8) and Lab 3 notes on where each VM runs.

YOU WILL PRODUCE

A site decision per tier, a shared-dependency table and a reduced-service list.

PROCEDURE

01 Decide what dr-b is for each tier: hot (running and in sync), warm (built, data restored periodically) or cold (an empty host).

02 Estimate how long each site type takes to serve users: provision, restore, configure, cut over.

03 Price a cloud region as an alternative site: idle cost versus cost during a recovery.

04 Trace dependencies prod-a and dr-b share: physical host, datastore, power, switch, ISP, DNS provider, identity provider, your laptop, and you.

05 For each shared dependency choose to break it, accept it, or document it as a known limit.

06 Plan reduced capacity: what runs on dr-b during a disaster and what waits.

WORKSHEET

DEPENDENCY	PROD-A USES	DR-B USES	SHARED?	DECISION

CHECK YOUR WORK

- [ ] Site type is consistent with each tier’s RTO.
- [ ] At least one shared dependency that was not obvious.
- [ ] A reduced-service list exists.

WATCH FOR

A recovery site on the same hypervisor, power circuit or cloud region. It passes the test and fails the disaster.

REFLECT

What does your recovery site still share with production?

What service would you knowingly run at reduced capacity during a disaster?

Build a cost justification

OBJECTIVE

Make an honest case for one tier of protection by comparing its recurring cost to the downtime it prevents.

YOU NEED

Strategy (Lab 8), risk register (Lab 7), BIA (Lab 6).

YOU WILL PRODUCE

A half-page funding request with every number traced to an assumption.

PROCEDURE

01 Choose one tier — Tier 1 is the usual choice.

02 Estimate downtime cost per hour: people blocked × hourly cost × share of work blocked, recovery labour, penalties. State reputational effects in words.

03 Estimate expected annual downtime without this protection, using your risk register’s likelihoods. Stay conservative.

04 Total the annual cost of the protection: storage, standby, tooling, testing time and maintenance.

05 Compare annual protection cost with expected annual loss avoided. Show your assumptions; do not inflate.

06 Note any regulatory or contractual driver, and the evidence an auditor would ask for.

07 Write the request: problem, option, cost, risk reduced, recurring commitment, owner.

WORKSHEET

LINE ITEM	ASSUMPTION	ANNUAL AMOUNT

CHECK YOUR WORK

- [ ] Every number has a stated assumption.
- [ ] Recurring costs are shown, not only setup.
- [ ] A program owner is named.

WATCH FOR

Multiplying a worst-case outage by certainty. Inflated numbers lose credibility the first time finance checks them.

REFLECT

Which assumption in your numbers is weakest?

Would your request still make sense if downtime cost were half your estimate?

PART III

Backup

Build backups that meet your RPO, survive an attacker holding your credentials, and are proven to restore.

Lab 12	Implement full-plus-incremental backups
Lab 13	Architect for 3-2-1 with an immutable copy
Lab 14	Prove an application-consistent backup restores
Lab 15	Configure an immutable backup and attack it
Lab 16	Run a full restore test against the clock

Implement full-plus-incremental backups

OBJECTIVE

Build a backup schedule that matches your RPO, and know exactly what each backup captures.

YOU NEED

The lab from Lab 3; your objectives from Lab 6.

YOU WILL PRODUCE

Scheduled full and incremental backups of the Gitea data, hourly database dumps and an inventory of what each contains.

PROCEDURE

01 Mount prod-a's second disk as /backup — a different medium from production.

```
sudo mkfs.ext4 /dev/sdb && sudo mkdir -p /backup
echo '/dev/sdb /backup ext4 defaults 0 2' | sudo tee -a /etc/fstab
sudo mount -a && sudo chown lab: /backup
```

02 Decide scope: the database, the Gitea volume (repositories, attachments, app.ini), compose.yaml and the host's own configuration. Write down what you exclude, and why.

03 Take a level-0 (full) archive of the Gitea volume. The .snar file records state so later runs are incremental.

```
V=/var/lib/docker/volumes/fieldapp_giteadata/_data
sudo tar --listed-incremental=/backup/gitea.snar \
  -czf /backup/gitea-full-$(date +%F-%H%M).tgz -C $V .
```

04 Take a full logical dump of the database.

```
docker compose exec -T db pg_dump -U gitea -Fc gitea \
  > /backup/db-$(date +%F-%H%M).dump
```

05 Make changes in Gitea: push a commit, upload an attachment, delete an issue. Run the tar command again with the same .snar file and a filename ending in -inc.

06 Inspect each archive with "tar -tzvf" and compare sizes. Confirm the incremental contains only what changed.

07 Wrap steps 3–4 in two scripts and schedule them. Before each weekly full, move the old .snar aside so the next run starts a new chain.

```
# crontab -e
0 * * * *   /home/lab/bin/backup-hourly.sh
30 2 * * 0  /home/lab/bin/backup-weekly-full.sh
```

08 Work out the worst-case data loss this schedule allows and compare it to your RPO. Record the gap if there is one.

CHECK YOUR WORK

- [ ] You can say what each archive contains without opening it.
- [ ] Worst-case loss from the schedule is at or under your RPO — or the gap is recorded.
- [ ] compose.yaml and host configuration are inside the backup scope.

WATCH FOR

Archiving the live database volume with tar and calling it a backup. Lab 14 shows why that may not restore.

REFLECT

What did your incremental backup capture that you did not expect?

Does your schedule truly meet your RPO at the worst moment of the hour?

Architect for 3-2-1 with an immutable copy

OBJECTIVE

Make sure no single failure or single compromised credential can take every copy at once.

YOU NEED

Backups from Lab 12, MinIO on vault, the MinIO client (mc) on your workstation.

YOU WILL PRODUCE

Three copies on two media with one off-site and one immutable, and an architecture diagram showing which credentials reach which copy.

PROCEDURE

01 Map what you have: production is copy 1; /backup on the second disk is copy 2, on a different medium. Neither is off-site.

02 On vault, create an ordinary bucket and one with object lock and a default retention.

```
mc alias set vault http://vault:9000 labadmin lab-only-secret
mc mb vault/dr-offsite
mc mb --with-lock vault/dr-immutable
mc retention set --default COMPLIANCE 14d vault/dr-immutable
```

03 Create a dedicated backup-writer user with a policy limited to these two buckets (mc admin user add, mc admin policy attach). Root credentials never go on prod-a.

04 Initialise restic repositories in both buckets and push the local backups and the app directory off-site.

```
export AWS_ACCESS_KEY_ID=backup-writer AWS_SECRET_ACCESS_KEY=<secret>
export RESTIC_PASSWORD_FILE=~/.restic-pass
for B in dr-offsite dr-immutable; do
  restic -r s3:http://vault:9000/$B init
  restic -r s3:http://vault:9000/$B backup /backup /home/lab/fieldapp
done
```

05 Store the restic password in two places that do not depend on prod-a: a password manager and a sealed printed copy.

06 Draw the architecture: each copy, its medium, its location and which credentials can reach it. Mark any single credential or host that can reach every copy.

07 Add both off-site pushes to the hourly backup script.

CHECK YOUR WORK

- [ ] Three copies, two media, one off-site, one immutable — each pointed to on the diagram.
- [ ] prod-a holds no root credentials for vault.
- [ ] The restic password is recoverable without prod-a.

WATCH FOR

One admin account that can delete production, the local backups and the off-site bucket. That is one copy with three addresses.

REFLECT

Which credential, if stolen, could reach the most copies?

Why does a second medium matter if both copies sit in the same building?

Prove an application-consistent backup restores

OBJECTIVE

See the difference between a crash-consistent copy and an application-consistent one, and prove the latter brings back a working service.

YOU NEED

The lab with the heartbeat running so the database is busy.

YOU WILL PRODUCE

Three database backups restored side by side on dr-b, with restore times and a working Gitea login from the good one.

PROCEDURE

01 Snapshot prod-a and dr-b.

02 Take a crash-consistent copy: archive the live data directory without stopping PostgreSQL.

```
sudo tar -czf /backup/pgdata-live.tgz \
  -C /var/lib/docker/volumes/fieldapp_pgdata/_data .
```

03 Take two application-consistent copies: a logical dump (as in Lab 12) and a physical base backup.

```
docker compose exec -T db pg_basebackup -U gitea -D /tmp/base -Ft -z -X stream
docker compose cp db:/tmp/base /backup/base-$(date +%F-%H%M)
```

04 Copy them to dr-b. Restore the dump into a fresh PostgreSQL container.

```
docker run -d --name t1 -e POSTGRES_PASSWORD=x postgres:16
docker exec t1 sh -c 'until pg_isready -q; do sleep 1; done; createdb -U postgres gitea'
docker exec -i t1 pg_restore -U postgres -d gitea --no-owner < db-latest.dump
```

05 Extract the crash-consistent archive into an empty volume and start PostgreSQL on it. Note whether it starts, what recovery messages appear in the log, and whether the data matches.

06 Prove working state: start Gitea on dr-b against the restored dump plus a restored Gitea volume. Log in and open a repository.

07 For each copy record restore time and the last heartbeat timestamp it contains.

CHECK YOUR WORK

- [ ] You can explain why the live tar did or did not start — and why you cannot rely on it either way.
- [ ] The dump restore ends with a working Gitea login.
- [ ] Restore times and heartbeat timestamps are recorded.

> **WATCH FOR**
>
> Concluding crash-consistent copies are fine because one started. Databases often recover from them — until the day they don’t. Application-aware methods are the guarantee.

REFLECT

Why might a crash-consistent copy start today and fail next month?

What does “working state” mean for an application rather than a database?

Configure an immutable backup and attack it

OBJECTIVE

Defend the backups as the attacker’s first target, and prove the immutable copy survives someone holding your backup credentials.

YOU NEED

The dr-immutable bucket from Lab 13.

YOU WILL PRODUCE

Evidence that an attack with backup-writer credentials — and an admin delete attempt — left recoverable data, plus a tamper check.

PROCEDURE

01 Confirm versioning and default retention are in force.

```
mc version info vault/dr-immutable
mc retention info --default vault/dr-immutable
```

02 Separate duties: backup-writer can write but cannot change retention; a separate vault-admin is kept off prod-a. Record who holds each.

03 Play the attacker on prod-a with backup-writer credentials and try to destroy history.

```
restic -r s3:http://vault:9000/dr-immutable forget --keep-last 1 --prune
mc alias set w http://vault:9000 backup-writer <secret>
mc rm --recursive --force w/dr-immutable
```

04 Observe: deletes may appear to succeed but only add delete markers. List versions to prove the data survives.

```
mc ls --versions --recursive vault/dr-immutable | head -20
```

05 As vault-admin, try to remove a locked version with --version-id. Compliance mode should refuse even you.

06 Recover a usable repository view from before the attack into a staging bucket, then check it.

```
mc mb vault/dr-staging
mc cp --recursive --rewind 30m vault/dr-immutable/ vault/dr-staging/
restic -r s3:http://vault:9000/dr-staging check
```

07 Confirm you can restore using only the off-prod-a copy of the restic password — encryption must never lock you out of your own backups.

08 Add a daily tamper check: alert if the snapshot count drops or the newest snapshot is older than your RPO.

CHECK YOUR WORK

- [ ] The attack left every locked version recoverable.
- [ ] The admin could not delete a locked version.
- [ ] restic check passes on the rewound copy.

WATCH FOR

Governance-mode retention with the root user stored on a backed-up host. An attacker who gets root simply lifts the lock.

REFLECT

What could an attacker with root on vault still do?

How would you know if your backups had been quietly tampered with?

Run a full restore test against the clock

OBJECTIVE

Turn a backup from a hope into evidence by restoring it — days later, from off-site, onto a clean host — and timing it.

YOU NEED

An off-site backup at least three days old; your Lab 9 procedure.

YOU WILL PRODUCE

A measured restore time and effective RPO, a corrected procedure and an automated weekly restore check.

PROCEDURE

01 Pick an off-site restic snapshot at least three days old. Record its ID and timestamp.

02 Revert dr-b to baseline. Start a stopwatch now — this measures your real RTO.

03 Follow your Lab 9 procedure exactly. Restore from vault onto dr-b.

```
restic -r s3:http://vault:9000/dr-offsite snapshots
restic -r s3:http://vault:9000/dr-offsite check --read-data-subset=10%
restic -r s3:http://vault:9000/dr-offsite restore <id> --target /restore
```

04 Restore the database and the Gitea volume, start the stack, then test as a user: log in, clone a repository, open an issue that existed when the backup was taken.

05 Stop the stopwatch when a user could work. Compare the newest heartbeat with the backup time to get the effective RPO of that backup.

```
psql -h dr-b -U gitea -d gitea -At -c "SELECT max(ts) FROM dr_heartbeat;"
```

06 Write down every deviation from the procedure, then fix the procedure.

07 Automate: a weekly job restores the newest dump into a scratch container and runs a known query. It fails loudly.

WORKSHEET

BACKUP ID	TAKEN AT	RESTORE START	SERVICE USABLE	ELAPSED	ISSUES

CHECK YOUR WORK

- [ ] Elapsed time is recorded and compared to RTO.
- [ ] A user-level check passed, not only a restore message.
- [ ] The procedure has been updated.

WATCH FOR

Counting “restore completed” as success. The test ends when a user can log in and do their work.

REFLECT

How did your measured RTO compare to the objective?

Which deviation from the procedure would have mattered most in a real event?

PART IV

Replication and Availability

Push RPO toward zero, and learn exactly what high availability does — and does not — protect you from.

Lab 17	Set up replication and measure the data-loss window
Lab 18	Show what high availability cannot save

Set up replication and measure the data-loss window

OBJECTIVE

Replicate the database to dr-b, then measure what asynchronous and synchronous modes really cost in data loss, latency and availability.

YOU NEED

Snapshots of prod-a and dr-b.

YOU WILL PRODUCE

Measured lag idle and under load, a recorded synchronous-mode stall, and a measured RPO from a hard failover.

PROCEDURE

01 On prod-a, create a replication role and allow dr-b to connect.

```
q "CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'lab-repl';"
docker compose exec db sh -c "echo 'host replication replicator <dr-b-ip>/32 scram-sha-256' \
  >> /var/lib/postgresql/data/pg_hba.conf"
q "SELECT pg_reload_conf();"
```

02 On dr-b, seed a standby with pg_basebackup -R and start it.

```
docker volume create pgreplica
docker run --rm -v pgreplica:/var/lib/postgresql/data -e PGPASSWORD=lab-repl \
  postgres:16 pg_basebackup -h prod-a -U replicator \
  -D /var/lib/postgresql/data -R -X stream -P
docker run -d --name replica -p 5432:5432 \
  -v pgreplica:/var/lib/postgresql/data postgres:16
```

03 Measure asynchronous lag from both ends.

```
psql -h dr-b -U gitea -d gitea -c "SELECT now() - max(ts) AS behind FROM dr_heartbeat;"
q "SELECT client_addr, state, sync_state, replay_lag FROM pg_stat_replication;"
```

04 Add distance: put 150 ms of latency on dr-b's interface, push a large repository, and measure again.

```
sudo tc qdisc add dev eth0 root netem delay 150ms
```

05 Switch to synchronous replication and watch commit latency on the heartbeat rise.

```
q "ALTER SYSTEM SET synchronous_standby_names = '*';"
q "SELECT pg_reload_conf();"
```

06 Stop the replica. In synchronous mode, writes on prod-a stall. Record it — that is the price of zero RPO with one standby. Revert to asynchronous.

07 Hard power-off prod-a mid-write. Promote the replica and compare its newest heartbeat with prod-a's (start prod-a isolated afterwards to read it). The difference is your measured RPO.

```
docker exec replica psql -U gitea -d gitea -c "SELECT pg_promote();"
```

CHECK YOUR WORK

- [ ] Asynchronous lag is measured idle and under load.
- [ ] The synchronous-mode stall is observed and logged.
- [ ] RPO from the hard failover is measured in seconds.

WATCH FOR

Leaving synchronous replication on with a single standby. When the link drops, production stops — you have traded data loss for an outage.

REFLECT

What RPO did you measure, and does it match your Tier 1 objective?

When would you accept the outage risk of synchronous replication?

Show what high availability cannot save

OBJECTIVE

Build a simple failover, watch it handle a component failure, then watch it faithfully replicate a disaster.

YOU NEED

The primary–replica pair from Lab 17 (re-seed if it was promoted).

YOU WILL PRODUCE

A timed failover and a two-column table of what HA handles versus what needs DR.

PROCEDURE

01 Make the database address configurable so the application can be repointed.

```
# compose.yaml, app service
      GITEA__database__HOST: ${DB_HOST}
# ~/fieldapp/.env
DB_HOST=db:5432
```

02 Write a failover script: promote the replica, repoint Gitea, restart it.

```
#!/bin/sh
set -e
ssh dr-b "docker exec replica psql -U gitea -d gitea -c 'SELECT pg_promote();'"
sed -i 's/^DB_HOST=.*/DB_HOST=<dr-b-ip>:5432/' ~/fieldapp/.env
docker compose -f ~/fieldapp/compose.yaml up -d app
```

03 Fail the component: “docker compose stop db”. Run the script and time it until Gitea works. This is HA doing its job. Then revert both VMs.

04 With the pair healthy again, have a “colleague” run a destructive statement on the primary.

```
q "DELETE FROM issue WHERE repo_id = 1;"
```

05 Check the replica within seconds: the deletion is already there. Failing over now gets you a faster copy of the damage.

06 Recover with DR: restore the last good backup (Lab 16 method). Measure how much data you lost. Lab 19 shows how to lose almost none.

07 Write a two-column table for your lab: failures HA handles, failures that need DR.

CHECK YOUR WORK

- [ ] Failover time is recorded.
- [ ] The deletion is confirmed on the replica.
- [ ] The DR column includes corruption, ransomware, site loss and human error.

WATCH FOR

Calling the replica “our DR”. It protects you from the server dying, not from anything the server is told to do.

REFLECT

Which of your earlier threats does HA handle, and which does it make worse?

How would you explain the HA–DR difference to a manager in two sentences?

PART V

Recovery Mechanics

Bring back data, systems, whole applications and cloud workloads, and recover a clean environment after ransomware.

Lab 19	Perform a point-in-time database recovery
Lab 20	Rebuild a host and restore its workload
Lab 21	Recover a multi-tier application end to end
Lab 22	Recover a workload into the cloud
Lab 23	Recover cleanly from ransomware

Perform a point-in-time database recovery

OBJECTIVE

Roll the database back to the second before a bad transaction, using a base backup and archived transaction logs.

YOU NEED

A clean prod-a and dr-b from snapshots.

YOU WILL PRODUCE

A recovered database containing the deleted rows and nothing after the bad transaction.

PROCEDURE

01 Mount /backup/wal into the db service at /wal (owned by uid 999, the postgres user in the image), then enable WAL archiving.

```
q "ALTER SYSTEM SET archive_mode = on;"
q "ALTER SYSTEM SET archive_command = 'test ! -f /wal/%f && cp %p /wal/%f';"
docker compose restart db
q "SELECT pg_switch_wal();"      # then: ls /backup/wal
```

02 Take a fresh base backup (Lab 14 command). Add the base and WAL directories to the hourly off-site push.

03 Create some activity. Then note the exact time and run the bad transaction. The heartbeat keeps writing afterwards, so precision matters.

```
q "SELECT now();"           # record as T_bad
q "DELETE FROM issue WHERE repo_id = 1;"
```

04 On dr-b, extract base.tar.gz into an empty data volume, mount the archived WAL at /wal, and set a recovery target just before T_bad.

```
touch recovery.signal
cat >> postgresql.auto.conf <<'EOF'
restore_command = 'cp /wal/%f %p'
recovery_target_time = '<T_bad minus 1s, with +00 offset>'
recovery_target_action = 'pause'
EOF
```

05 Start PostgreSQL on dr-b. Watch the log until it reports that the recovery target was reached and replay is paused.

06 Verify before promoting: the deleted issues exist and the newest heartbeat is just before T_bad. If you overshot, wipe and retry with an earlier target. Then end recovery.

```
psql -h dr-b -U gitea -d gitea -c "SELECT pg_wal_replay_resume();"
```

07 Decide how to use it: switch service to the recovered database, or extract just the deleted rows and re-insert them into production. Write when you would choose each.

CHECK YOUR WORK

- [ ] The recovered database contains the deleted rows.
- [ ] No heartbeat exists after T_bad; the gap is measured in seconds.
- [ ] You recorded the reasoning for switch versus extract.

WATCH FOR

Giving the recovery target in local time when the server runs in UTC. Always include the time-zone offset.

REFLECT

How precise did your recovery target need to be, and why?

When is extracting rows better than switching to the recovered database?

Rebuild a host and restore its workload

OBJECTIVE

Recover a whole system two ways — restoring an image and rebuilding from code — and learn when each is the right call.

YOU NEED

Hypervisor access; a Git repository that does not live on prod-a.

YOU WILL PRODUCE

Two successful recoveries of prod-a, a bootstrap script that reproduces it, and a written rebuild-versus-restore rule.

PROCEDURE

01 Write prod-a as code: a bootstrap script or cloud-init file that installs packages, mounts disks, creates users, writes compose.yaml and fetches secrets from your offline store. Commit it somewhere other than prod-a — and note why the lab's own Gitea does not count.

02 Image path: back up prod-a's whole VM disk with your hypervisor's tool (vzdump, VBoxManage clonemedium, or similar). Record size and time.

03 Declare prod-a destroyed. Delete the VM.

04 Path A: restore the image to a new VM. Time to boot, and time to working service.

05 Path B: create a blank Ubuntu VM, run the bootstrap, then restore data from off-site.

```
scp bootstrap.sh new-host: && ssh new-host 'sudo bash bootstrap.sh'
restic -r s3:http://vault:9000/dr-offsite restore latest --target /restore
```

06 Check the configuration that makes a restored system actually work: hostname, IP, firewall, time sync, TLS, app.ini, Gitea ROOT_URL. Move every hand fix into the bootstrap.

07 Compare paths A and B: speed, confidence that the system is clean, and what each carries over — including problems.

CHECK YOUR WORK

- [ ] Both paths reached a working Gitea.
- [ ] Every manual fix now lives in code.
- [ ] You know which path you would use after ransomware, and which after hardware loss.

WATCH FOR

Infrastructure code stored only inside the environment it is meant to rebuild.

REFLECT

Which path — image or rebuild — gave you more confidence, and why?

What configuration did you only discover by rebuilding?

Recover a multi-tier application end to end

OBJECTIVE

Bring back a complete service — proxy, application, database, secrets and integrations — in the right order, and prove it from the user's side.

YOU NEED

The lab plus a self-signed or internal-CA certificate for git.lab.

YOU WILL PRODUCE

A dependency map, a recovery order with checks per tier, and a passed end-to-end user test from outside dr-b.

PROCEDURE

01 Add a tier: an nginx reverse proxy with TLS in front of Gitea. Users now go to https://git.lab; update Gitea's ROOT_URL to match.

```
proxy:
  image: nginx:1.27
  ports: ["443:443"]
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    - ./certs:/etc/nginx/certs:ro
  depends_on: [app]
```

02 Draw the dependency web: DNS name → proxy → certificate → Gitea → app.ini secrets → database → credentials, plus SSH keys and webhook targets.

03 Write the recovery order with a check after each tier: database accepts connections; Gitea starts and connects; proxy serves a valid certificate; the name resolves to dr-b.

04 Revert dr-b to baseline and recover the whole stack in that order from off-site backups.

05 Recover once deliberately with a fresh app.ini instead of the original. Note what breaks — sessions, two-factor, stored secrets — then restore the real one.

06 Test as a user from a machine that is not dr-b: browse to https://git.lab, log in, clone over HTTPS and SSH, push a commit, trigger a webhook.

07 Record every "started but not working" moment and the hidden dependency behind it.

CHECK YOUR WORK

- [ ] The end-to-end user test passed from outside dr-b.
- [ ] A secrets inventory lists every secret and where it is kept.
- [ ] The recovery order is documented in the plan.

WATCH FOR

Declaring success from the server console. A healthy container says nothing about whether users can reach it by name with a valid certificate.

REFLECT

What was your most surprising “started but not working” moment?

Which secret, if lost, would make recovery impossible?

Recover a workload into the cloud

OBJECTIVE

Use the cloud as a pay-when-needed recovery site, and find out what the provider protects and what stays yours.

YOU NEED

A cloud account with object storage and VMs, and a budget alert set before you begin.

YOU WILL PRODUCE

A recovery of the lab into a cloud VM built from code, its timing, and a standby cost comparison.

PROCEDURE

01 Set the budget alert. Create a bucket in a region you do not otherwise use; turn on versioning and object lock if the provider offers them.

02 Copy your off-site restic repository to the cloud bucket.

```
restic -r s3:https://<endpoint>/<bucket> init \
  --from-repo s3:http://vault:9000/dr-offsite --copy-chunker-params
restic -r s3:https://<endpoint>/<bucket> copy \
  --from-repo s3:http://vault:9000/dr-offsite
```

03 Read the provider’s shared-responsibility statement. Write what they protect and what remains yours: data, configuration, identity and backups.

04 Turn your Lab 20 bootstrap into user-data for the provider’s VMs, with the region as a parameter.

05 Declare the lab site lost. Launch the VM from code, restore from the cloud bucket, and point git.lab at it. Time it.

06 Run the Lab 21 user test against the cloud copy.

07 Tear everything down. Record what standby costs when idle (storage only) and what the test hours cost. Compare with keeping dr-b warm.

08 Confirm the bucket, VM image and any keys are available in the recovery region — not only in the region you happened to use.

CHECK YOUR WORK

- [ ] The recovery was launched from code, not by clicking.
- [ ] Idle and active costs are recorded.
- [ ] All test resources are deleted.

> **WATCH FOR**
>
> Backups in the same region and account as production. A region outage or a compromised account takes both.

REFLECT

What did you assume the cloud provider protected before reading their statement?

How does pay-when-needed standby change your Lab 10 site choice?

Recover cleanly from ransomware

OBJECTIVE

Simulate an attack that lingers, encrypts data and goes after backups — then recover to a clean point without bringing the malware back.

YOU NEED

All three VMs snapshotted; the immutable repository holds a snapshot more than an hour old.

YOU WILL PRODUCE

A clean recovery from the immutable copy, a compromise-to-service timeline and a record of rotated credentials.

PROCEDURE

01 Simulate dwell time: plant persistence and a marker, then wait for at least one backup cycle so the infection reaches your recovery points.

```
echo '*/10 * * * * root /usr/local/bin/.updater' | sudo tee /etc/cron.d/sysupdate
sudo touch /var/lib/docker/volumes/fieldapp_giteadata/_data/.ioc-marker
```

02 Detonate: encrypt repository packs, wipe local backups and use the stolen writer keys against both off-site repositories.

```
cd /var/lib/docker/volumes/fieldapp_giteadata/_data/git/repositories
sudo find . -name '*.pack' -exec sh -c \
  'openssl enc -aes-256-cbc -pbkdf2 -pass pass:x -in "$1" -out "$1.locked" && rm "$1"' _
{} \;
rm -rf /backup/*
for B in dr-offsite dr-immutable; do
  restic -r s3:http://vault:9000/$B forget --keep-last 1 --prune
done
```

03 Isolate: disconnect prod-a's network at the hypervisor — do not shut it down, so evidence is preserved. Open an incident log; response and recovery now run in parallel.

04 Assess: which copies survived? Establish when the compromise began — the earliest indicator, such as the marker or cron file timestamps — not when encryption started.

05 Pick a recovery point before the compromise. Rewind the immutable bucket into staging, restore to a quarantine directory, and search for the marker.

```
mc cp --recursive --rewind 3h vault/dr-immutable/ vault/clean-staging/
restic -r s3:http://vault:9000/clean-staging restore latest --target /quarantine
for f in /quarantine/backup/gitea-*.tgz; do
  tar -tzf "$f" | grep -q ioc-marker && echo "INFECTED: $f"
done
```

06 Recover onto a new host built from code (Lab 20) — never onto the compromised one. Rotate every credential the attacker could have seen: database, restic and backup-writer keys, Gitea admin, SSH keys.

07 Verify before reconnecting: no marker, no unknown cron entries, no unknown Gitea admins or deploy keys, then the user test. Only then restore network access.

CHECK YOUR WORK

- [ ] The recovery came from the immutable copy and the marker search returned nothing.
- [ ] Every credential on your list was rotated.
- [ ] A timeline from compromise to restored service is written.

WATCH FOR

Restoring the latest backup. After ransomware, latest often means infected. Choose the recovery point from the compromise time, not the detonation time.

REFLECT

How far back did you need to go, and how did you decide?

What would have happened if you had restored onto the compromised host?

PART VI

Testing and Sustaining

Turn the plan into a proven capability, learn from every gap, keep it current, and account for people and the wider business.

Lab 24	Run a full failover test
Lab 25	Fix the gaps and retest
Lab 26	Change the environment, then the plan
Lab 27	Run a people-first tabletop exercise
Lab 28	Extend the plan into continuity

Run a full failover test

OBJECTIVE

Execute the whole plan on the recovery infrastructure and measure actual RTO and RPO against your objectives.

YOU NEED

Your complete plan, an observer or timekeeper if possible, the heartbeat running.

YOU WILL PRODUCE

Measured RTO and RPO compared with objectives, and a gap list written while memory is fresh.

PROCEDURE

01 Plan the test: scope (full service), success criteria (Tier 1 RTO and RPO from Lab 6), timekeeper, abort criteria and rollback.

02 Run a 30-minute tabletop of the same scenario first to catch the obvious gaps cheaply.

03 Start the clock: power off prod-a in the hypervisor and record T0.

04 Execute the plan exactly as written on dr-b. The timekeeper logs every step, delay and deviation. Do not fix the plan mid-test — log instead.

05 Declare service restored only when the user test passes. Record T1. Actual RTO = T1 − T0.

06 Actual RPO = T0 − the newest heartbeat in the recovered database.

```
psql -h dr-b -U gitea -d gitea -At -c "SELECT max(ts) FROM dr_heartbeat;"
```

07 Run on dr-b for at least an hour of real use. Fail back now, or record that failback is deferred to Lab 29.

08 Write the gap list immediately.

WORKSHEET

GAP	WHAT HAPPENED	PLAN SAID	EFFECT ON RTO/RPO	OWNER

CHECK YOUR WORK

- [ ] Measured RTO and RPO sit next to their objectives in one table.
- [ ] Every deviation is logged.
- [ ] Nothing was pre-staged or tidied before the test.

WATCH FOR

Pre-staging the recovery the day before so the test looks clean. You will have measured a rehearsal, not a recovery.

REFLECT

Which gap cost the most time?

How close were your measured RTO and RPO to the estimates from Lab 4?

Fix the gaps and retest

OBJECTIVE

Make the test count: trace each gap to a root cause, change the plan, and prove the fix worked.

YOU NEED

The test log and gap list from Lab 24.

YOU WILL PRODUCE

A versioned plan with changes logged, a retest result per gap and the start of a metrics trend.

PROCEDURE

01 Gather the test log, timestamps and gap list.

02 For each gap, write what happened against what the plan said, without assigning blame.

03 Ask why until you reach something you can change: a missing step, missing access, a wrong objective, an architecture limit.

04 Decide where each fix belongs: the procedure, the strategy (Lab 8) or the architecture — replication, a new site, an extra copy.

05 Make the changes. Version the plan and log what changed and why.

06 Retest the affected parts first, then repeat the full failover within two weeks.

07 Start a trend table: measured RTO, measured RPO and number of gaps per test.

WORKSHEET

GAP	ROOT CAUSE	FIX	LANDS IN	RETEST RESULT

CHECK YOUR WORK

- [ ] Every gap has a fix, or an accepted risk with an owner.
- [ ] Retests show the fixes work.
- [ ] The trend table has two data points.

WATCH FOR

Writing “be more careful” as a fix. If the fix is a person trying harder, the gap will be back.

REFLECT

Which root cause appeared behind more than one gap?

Which fix changed architecture rather than procedure?

Change the environment, then the plan

OBJECTIVE

Experience plan decay first-hand: make a significant change, find every place it breaks the plan, and retest.

YOU NEED

A working, tested lab and plan.

YOU WILL PRODUCE

An updated plan, a successful retest of the changed component and a DR checklist for future changes.

PROCEDURE

01 Make a significant change: upgrade PostgreSQL 16 to 17 via dump and restore, move Gitea data to a new volume, or add a service such as a CI runner.

02 Before calling it done, ask of the change: does it alter the inventory, dependencies, backup scope, restore steps, objectives, credentials or contacts?

03 Update the plan: inventory, dependency diagram, procedures and backup scripts.

04 Remove procedures for anything you decommissioned.

05 Retest — at minimum a restore of the changed component, preferably a partial failover.

06 Add the DR questions from step 2 to your change process so they run every time.

07 Set a review cadence: contacts monthly, procedures quarterly, BIA yearly. Put the dates in a calendar.

08 Check the printed and off-site copies of the plan match the new version.

WORKSHEET

CHANGE	PLAN SECTIONS AFFECTED	UPDATED	RETESTED	NOTES

CHECK YOUR WORK

- [ ] The restore of the changed component succeeded.
- [ ] Obsolete procedures are gone.
- [ ] The next review date is scheduled.

WATCH FOR

PostgreSQL 17 cannot start on version 16 data files. A procedure that restores an old base backup into a new container fails exactly when you need it.

REFLECT

Which part of the plan would you have forgotten to update without the checklist?

How often does your real environment change compared with your review cadence?

Run a people-first tabletop exercise

OBJECTIVE

Test communication and decision authority under pressure — where technically sound plans most often fail.

YOU NEED

Two to six people playing roles, plus a facilitator. Solo: play each role in turn and time yourself.

YOU WILL PRODUCE

A decision log, an updated authority table with deputies, a communication tree and message templates.

SCENARIO

Friday, 17:30. Ransomware has hit. Company chat and email sign in through the affected identity provider. The technical lead's phone goes to voicemail.

PROCEDURE

01 Assign roles: incident lead, technical recovery, communications, business owner, executive. The facilitator holds the injects.

02 Brief the scenario. Participants may only use channels listed in the plan's contact sheet.

03 Play an inject every ten minutes: a customer posts publicly about the outage; the executive asks whether to pay; restore will take nine hours; legal asks about notification deadlines; the backup lead resigns mid-exercise.

04 Track decisions: who declared the disaster, who authorised failover, how long each took, who waited on whom.

05 Debrief: decisions with no clear owner, messages that never arrived, and moments people read the plan for the first time.

06 Update the plan: decision-authority table with deputies, a communication tree, and message templates for staff, customers and leadership.

WORKSHEET

TIME	INJECT	DECISION / ACTION	WHO DECIDED	CHANNEL USED

CHECK YOUR WORK

- [ ] Every decision had an empowered owner — or now has one.
- [ ] Communication worked without the primary channels.
- [ ] Each role has a named deputy.

WATCH FOR

Letting the most senior person answer everything. Test whether the plan works when they are absent.

REFLECT

Which decision took longest, and what slowed it?

Who turned out to be a single point of failure?

Extend the plan into continuity

OBJECTIVE

Find the non-technical dependencies your recovery assumes, and keep the business running while technology comes back.

YOU NEED

Your BIA (Lab 6) and current plan.

YOU WILL PRODUCE

A continuity view: non-technical dependencies, manual workarounds with time limits, and how DR fits beside incident response and crisis management.

PROCEDURE

01 For each Tier 1 process ask: if the technology were restored and nothing else changed, could the business function?

02 List non-technical dependencies: people and skills, workspace, suppliers (ISP, cloud, SaaS, payroll), physical items (hardware tokens, keys) and information (contracts, phone numbers).

03 Write a manual workaround for each Tier 1 process: developers work locally and push later; releases freeze; issues go to a shared document.

04 Check suppliers: what is their DR, and what is your recourse if they are the disaster?

05 Plan people and place: where staff work if the office or region is unavailable, and who can approve emergency spending.

06 Check that each DR RTO fits inside how long its workaround can last.

07 Map who leads when: incident response, crisis management, continuity and DR.

WORKSHEET

DEPENDENCY	TYPE	OWNER	WORKAROUND	MAX DURATION

CHECK YOUR WORK

- [ ] Each Tier 1 process has a workaround with a time limit.
- [ ] At least two supplier dependencies are listed.
- [ ] Every RTO is shorter than its workaround’s limit.

WATCH FOR

Recovering Gitea in four hours while nobody can sign in because the single sign-on provider is also down.

REFLECT

Which non-technical dependency would stop the business even after full recovery?

How long can each workaround realistically last?

PART VII

Execution and Mastery

Run a real recovery under pressure, diagnose recoveries that fail, audit your program, and plan what comes next.

Lab 29	Execute a full recovery from a declared disaster
Lab 30	Diagnose staged recovery failures
Lab 31	Audit your program against the failure catalog
Lab 32	Assess your skills and plan the next 90 days

Execute a full recovery from a declared disaster

OBJECTIVE

Bring the course together: detect, declare, recover in order, verify each stage, communicate, and fail back safely.

YOU NEED

A partner to trigger an unannounced disaster, or a random pick from: corrupted database, deleted VM, ransomware simulation, site loss.

YOU WILL PRODUCE

A recovery inside (or measured against) your Tier 1 RTO, a full timeline, a safe failback and a post-incident review.

PROCEDURE

01 Have your partner choose and trigger the disaster without telling you which. Your time limit is the Tier 1 RTO.

02 Detect and assess: what is broken, since when, and what is still trustworthy. Log the detection time.

03 Apply your activation criteria and declare formally in the log. Any hesitation counts against RTO — note it.

04 Prioritise by tier, recover in dependency order, and verify each stage before starting the next.

05 Send status updates on the schedule your plan defines, even when the update is “no change”.

06 Give the all-clear only after the user test passes and a 30-minute watch period ends.

07 Fail back once prod-a is rebuilt: treat it as a planned change — sync data back from dr-b, schedule a window, verify, and keep dr-b ready to resume.

08 Write the post-incident review in the Lab 25 format.

WORKSHEET

TIME	EVENT / ACTION	STAGE	VERIFIED?	NOTES

CHECK YOUR WORK

- [ ] Recovery finished within RTO — or you know by how much it missed, and why.
- [ ] No stage was skipped without verification.
- [ ] Failback lost no data: the heartbeat is continuous.

WATCH FOR

Failing back the moment prod-a boots. Failback is a second migration with live data; give it the same discipline.

REFLECT

Where did pressure push you to skip a verification?

What made the activation decision clear or unclear?

Diagnose staged recovery failures

OBJECTIVE

Practise finding which layer of a recovery is failing, with evidence, before changing anything.

YOU NEED

A partner to stage failures one at a time, or run them yourself in random order.

YOU WILL PRODUCE

A diagnosis log for each failure and a one-page recovery troubleshooting checklist.

PROCEDURE

01 Draw your diagnostic map: backup or replica → target infrastructure → data → application → dependencies. Write the first check for each layer.

02 Have your partner stage one failure at a time, then attempt a normal recovery onto dr-b.

```
F1  corrupt backup:   truncate -s 100K /backup/db-latest.dump
F2  missing secret:   remove the DB password from dr-b's .env
F3  out of order:     start Gitea before the database is restored
F4  wrong identity:   ROOT_URL still points at prod-a
F5  broken TLS:       remove the certificate from the proxy
F6  locked out:       restic password file absent on the recovery host
```

03 For each, write the symptom first, then name the layer — before touching anything.

04 Collect evidence: restore output, container logs, PostgreSQL log, proxy log, browser error. Paste it into your log; do not rely on scrollback.

05 When a layer fails, check its prerequisites: is the thing it depends on actually recovered?

06 Set a fallback rule with a time box: when to abandon this recovery point or method for an older backup, the replica or the image path.

07 Turn your notes into a one-page checklist you could follow while everything is on fire.

WORKSHEET

FAILURE	SYMPTOM	LAYER	EVIDENCE	ROOT CAUSE	FIX

CHECK YOUR WORK

- [ ] Each failure was attributed to the right layer before any fix.
- [ ] A time-boxed fallback rule exists.
- [ ] The checklist fits on one page.

WATCH FOR

Changing three things at once when the service will not start. You may fix it and never know which change mattered — or break something else.

REFLECT

Which failure was hardest to attribute to a layer?

When should you abandon a recovery path and fall back?

Audit your program against the failure catalog

OBJECTIVE

Check, with evidence, that each classic way recoveries fail has been designed out of your lab program.

YOU NEED

Every artefact from the previous labs.

YOU WILL PRODUCE

A completed audit, remediated gaps with retests, and a one-paragraph status a manager could read.

PROCEDURE

01 For each failure mode in the worksheet, find evidence that it is designed out: a test record, a configuration, a document.

02 Treat “we think so” as a gap.

03 Prioritise gaps using your risk register.

04 Remediate each gap and link the fix to the lab where you practised it.

05 Re-run the relevant test for each fix.

06 Summarise the program’s status in one paragraph.

WORKSHEET

FAILURE MODE	EVIDENCE	GAP?	REMEDIATION	DONE
Untested backup				
Recovery point too old				
Missing dependency				
Recovered but not working				
Compromised backup				
Out-of-date plan				
One person who knows how				
Found in disaster, not test				

CHECK YOUR WORK

- [ ] Every row has evidence or a remediation.
- [ ] Retests are complete.
- [ ] The summary is written.

WATCH FOR

Auditing the documents rather than the capability. A current plan that has never been executed still fails the first row.

REFLECT

Which failure mode had the weakest evidence?

Which gap had you believed was already closed?

Assess your skills and plan the next 90 days

OBJECTIVE

Measure what you can now do, choose where to deepen, and commit to building or maturing a real DR program.

YOU NEED

Your lab log and every artefact from this guide.

YOU WILL PRODUCE

A skills self-assessment backed by artefacts, a lab maintenance schedule and a 90-day program plan.

PROCEDURE

01 Rate yourself 1–4 in each area: analysis (BIA and risk), strategy and sites, backup, replication and HA, recovery mechanics (data, systems, applications, cloud, cyber), testing and learning, people and continuity.

02 Back each rating with an artefact from this guide. No artefact, lower the rating.

03 Choose a specialisation — business continuity, cyber recovery, cloud DR or DR architecture — and the framework or certification that matches it, such as ISO 22301 for continuity or NIST SP 800-34 for contingency planning.

04 Keep the lab alive: decide which checks run monthly (restore test, immutability check) so the skills stay current.

05 Write a 90-day plan for a real program. Days 1–30: inventory, BIA, one restore test. Days 31–60: strategy, 3-2-1 with an immutable copy, a first procedure. Days 61–90: tabletop, partial failover, review cadence.

06 Share the plan with someone who will hold you to it.

WORKSHEET

PERIOD	GOAL	DELIVERABLE	EVIDENCE OF DONE
Days 1–30			
Days 31–60			
Days 61–90			

CHECK YOUR WORK

- [ ] Each rating is backed by an artefact.
- [ ] Each 90-day period has a concrete deliverable.
- [ ] The lab maintenance schedule is in your calendar.

WATCH FOR

Planning to learn more tools. The durable skill is the RTO-and-RPO spine and proving recovery by test; the tools will change.

REFLECT

Which area of the course is your strongest, and which needs the most practice?

What is the first deliverable you will produce in a real organisation?
